

Day 4:
OOPS:

* Instead of writing everything inside function, we create objects, that contains both data and behaviour.

* variable & ^{contain} methods: , actions
↓
data

<u>Car variable</u>	<u>methods</u>
colour	start
brand	stop
model	Accelerate
shape	

stored in blue print

Blue print $\rightarrow$ class

class $\rightarrow$ Blue print

Object $\rightarrow$ Real instance.
that created from
blue print.

eg:

(class student) $\rightarrow$ blue print

pass

s1 = student()

s2 = student()

print(s1)

print(s2)

$\rightarrow$ object

mobile design

Construction:

⇒ `--init--()`

* Whenever we create a object python automatically called special method its called construction

`self` → build keyword, it denotes current objects.

```
class student:
```

```
    def __init__(self):
```

```
        print("Construction called")
```

```
s1 = student()
```

output:

Construction called

why construction?

To initialize object data!

Instance Variable & Method:

IV → Belongs to each object

```
class student:
```

```
    def __init__(self, name, age):
```

```
        self.name = name
```

```
        self.age = age
```

```
s1 = student("ram", 20)
```

```
print(s1.name)
```

```
class student:
```

```
    def __init__(self, name):
```

```
        self.name = name
```

```
    def display(self):
```

```
        print(self.name)
```

```
s1 = student("ram", 20)
```

```
print(s1.name)
```


Variable 3 type:

- Instance Variable → each object separate value
- class Variable → shared by an object
- Local Variable → inside method access

Pillars of oops:

- * Inheritance
- * Polymorphism
- * Encapsulation
- * Abstraction

Inheritance:

One class acquires properties of another class

It allows one class to reuse the property.

ex:

```
class animal:
    def eat(self):
        print(eating):

class dog (animal)
    dog bark (self):
        print ("barking")
```

```
d = dog()
d.eat()
d.bark()
```

output,

eating
barking

Types of inheritance

- * single
- * Multiple
- * Multi level
- * Hybrid
- * Hierarchy

Polymorphism:

- * child classes Replaces the parent classes. (over writing)
- * one method to many

2 Method:

Method

Overloading

Method

Overwriting

Overwriting:

child class ^{Replace} the parent method

~~class animal~~
~~def sound (self):~~

overloading: $\rightarrow$ python doesn't support directly

over levels inside

```
def add(a, b, c = 0):  
    return a + b + c
```

Encapsulation:

Data hiding $\rightarrow$ protect data from direct access

Abstraction:

Hides the code implementation
show only required functionality

Eg:

ATM $\rightarrow$ can show acc. balance but not other functionality.

eg (2)

and you drive a car using the steering wheel
the peddles so you don't need to know
the engine works.